So far, we've been dealing with a fairly small set of resources. On a larger system there may be types of objects for which a `review` may be appropriate or to which a `who_bought` action might reasonably be applied. Instead of repeating these instructions for each resource, consider refactoring your routes using `concern`s to capture the common behavior.

```
concern :reviewable do
  resources :reviews
end

resources :products, concern: :reviewable
resources :users, concern: :reviewable
```

The preceding definition of the `products` resource is equivalent to the one in the previous section.

Shallow Route Nesting

At times, nested resources can produce cumbersome URLs. A solution to this is to use shallow route nesting:

```
resources :products, shallow: true do
  resources :reviews
end
```

This will enable the recognition of the following routes:

```
/products/1          => product_path(1)
/products/1/reviews => product_reviews_index_path(1)
/reviews/2           => reviews_path(2)
```

Try the `rails routes` command to see the full mapping.

Selecting a Data Representation

One of the goals of a REST architecture is to decouple data from its representation. If a human uses the URL path `/products` to fetch products, they should see nicely formatted HTML. If an application asks for the same